ENTERPRISE ARCHITECTURE & OPERATIONS SPECIFICATION

CA Agent 2.0: Full Production Deployment Guide

End-to-End Walkthrough: Architecture, Linux VPS / Systemd, Docker Microservices, Cloud PaaS, Nginx & Hardening

Platform: CA Agent 2.0 (Chartered Accountant AI)	**Date:** October 2026	**Stack:** React 18, Vite, FastAPI, Ollama, Gemini
Target Environments: Ubuntu 22.04/24.04, Docker, Vercel/Render	**Roadmap:** Step 11 & 12 Production Rollout	**Security:** TLS 1.3, Scoped CORS, Non-Root Daemons

1. System Architecture & Hardware Sizing

CA Agent 2.0 is an enterprise chartered accountant copilot designed for Indian taxation, GST, bank statement anomaly detection, and guided ITR filing. Deploying the system requires coordinating four coordinated tiers: the **Client Frontend**, the **FastAPI Application Server**, the **Ollama Vector Embedding Engine**, and the **Google Gemini Multimodal API**.

Tier	Service & Framework	Default Port	Role & Resource Footprint
Frontend	React 18 + Vite + Tailwind + Recharts	`8080 (Dev)` `80 / 443 (Prod)`	Static SPA bundle served via Nginx. Handles chat sessions, multi-document financial dashboards, and guided ITR wizard.
Backend API	FastAPI (Python 3.11/3.13) + Uvicorn	`8000`	Orchestrates RAG retrieval, CrossEncoder reranking, IsolationForest anomaly detection, memory compression, and ITR state machine.
Vector Engine	Ollama (bge-m3:latest)	`11434`	Generates 1024-dimensional multilingual embeddings for query matching against ~270 MB ICAI corpora. Requires ~1.5 GB RAM.
Cognitive API	Google Gemini 2.5 Flash / 1.5 Pro	`HTTPS (443 Outbound)`	Statutory CA reasoning, document vision extraction (PDF/Images), and conversational memory compression.

Hardware Sizing Recommendation

- **Compute**: 4 vCPU minimum (e.g. AWS t3.xlarge / c6i.xlarge, DigitalOcean 8GB Droplet, Hetzner CPX41).
- **Memory**: 8 GB RAM minimum; **16 GB RAM strongly recommended** to hold all three ICAI joblib matrices and Ollama bge-m3 in RAM without swapping.
- **Disk**: 40 GB NVMe SSD (Ollama model ~2.5 GB, ICAI corpora ~300 MB, PyTorch & HuggingFace model cache ~2 GB).
- **Network**: Outbound HTTPS connectivity to *generativelanguage.googleapis.com:443*.

2. Environment Variables & Asset Pre-Flight Check

Configure production environment variables and verify that all precomputed serialized model assets exist in the root working directory.

Variable Name	Scope	Required	Description & Production Setting
`GEMINI_API_KEY`	Backend	Yes	Primary Gemini API key used by ca_agent.py and file_handler.py (Vision parsing).
`DEV_GEMINI_API_KEY`	Backend	Yes	Secondary key used for HyDE query expansion (hyde.py) and ITR agent (itr_agent.py).
`OLLAMA_URL`	Backend	Optional	Address of Ollama instance. Default: http://localhost:11434 (or http://ollama:11434 in Docker).
`ENABLE_HYDE`	Backend	Optional	false for lowest latency; true for advanced hypothetical document generation.
`ENABLE_METRICS`	Backend	Optional	Must be set to false in production to eliminate RAGAS judge overhead.
`VITE_API_URL`	Frontend	Yes	Target API base for React app: https://api.yourdomain.com or /api when proxied.

Critical Pre-Flight Check: Joblib Corpora Files

The backend server loads 5 serialized files directly on startup in search.py. If any file is missing, the server will crash with *RuntimeError*:

- embeddings_foundation.joblib (88 MB)
- embeddings_Intermediate.joblib (76 MB)
- embeddings_Final.joblib (102 MB)
- graphRag/income_tax_graph.gpickle (85 KB)
- graphRag/income_tax_embeddings.joblib (4.5 MB)

Rule: When cloning via Git, ensure **Git LFS** is installed (git lfs install && git lfs pull) or transfer these files via secure S3/rsync.

3. Method 1: Bare-Metal / Linux VPS Deployment (Ubuntu 22.04 / 24.04)

This is the production-grade approach for a dedicated cloud virtual machine (AWS EC2, DigitalOcean, Hetzner, Linode). It provides maximum throughput, native systemd process supervision, local Ollama caching, and TLS termination via Nginx.

Step 3.1: Server Provisioning & Toolchain Setup

```
# Update system packages and install dependencies
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-venv git git-lfs curl nginx certbot \
                    python3-certbot-nginx build-essential ufw
# Install Node.js 20 LTS (required for building the React frontend)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
# Verify runtimes
python3 --version  # Expect Python 3.11+
node -v            # Expect v20.x+
git lfs install    # Enable Large File Storage
```

Step 3.2: Ollama Service Setup & bge-m3 Model Pull

Install Ollama as a managed background systemd daemon and pull the multilingual dense embedding model:

```
# Download and run official Ollama installer
curl -fsSL https://ollama.com/install.sh | sh
# Enable and start Ollama service
sudo systemctl enable ollama
sudo systemctl start ollama
# Pull the bge-m3 embedding model (approx 2.3 GB)
ollama pull bge-m3
# Verify model is available locally
curl http://localhost:11434/api/tags
```

Step 3.3: Repository Setup & Python Virtual Environment

```
# Prepare application directory
sudo mkdir -p /var/www/ca-agent && sudo chown -R $USER:$USER /var/www/ca-agent
cd /var/www/ca-agent
# Clone repository
git clone <YOUR_GIT_REPO_URL> .
git lfs pull   # Fetch the 270 MB .joblib corpora files
# Create and activate virtual environment
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
# Create temporary upload directory with write permissions
mkdir -p temp && chmod 775 temp
# Create production .env file
cat <<EOF > .env
GEMINI_API_KEY="YOUR_ACTUAL_GEMINI_API_KEY"
DEV_GEMINI_API_KEY="YOUR_ACTUAL_DEV_GEMINI_API_KEY"
ENABLE_HYDE=false
ENABLE_METRICS=false
OLLAMA_URL="http://localhost:11434"
EOF
```

Step 3.4: Configure Backend Systemd Daemon

Create a systemd unit at /etc/systemd/system/ca-backend.service to manage Uvicorn workers and auto-restart on failure:

```
sudo tee /etc/systemd/system/ca-backend.service > /dev/null <<'EOF'
[Unit]
Description=CA Agent 2.0 FastAPI Backend
After=network.target ollama.service
Wants=ollama.service
[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/ca-agent
EnvironmentFile=/var/www/ca-agent/.env
ExecStart=/var/www/ca-agent/.venv/bin/uvicorn api:app --host 127.0.0.1 --port 8000 --workers 2 --timeout-keep-alive 65
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
LimitNOFILE=65536
[Install]
WantedBy=multi-user.target
EOF
# Set ownership, reload systemd daemon, and launch service
sudo chown -R www-data:www-data /var/www/ca-agent
sudo systemctl daemon-reload
sudo systemctl enable ca-backend
sudo systemctl start ca-backend
# Check service status
sudo systemctl status ca-backend --no-pager
```

Step 3.5: Build and Deploy the React Frontend

Compile the frontend SPA into static production assets in /var/www/ca-agent/dist:

```
cd /var/www/ca-agent
# Set production API base (use relative path if proxying via Nginx)
echo "VITE_API_URL=/api" > .env.production
# Install npm dependencies and build production bundle
npm ci
npm run build
# Verify build output
ls -lh dist/index.html
```

Step 3.6: Configure High-Performance Nginx Reverse Proxy & SSL

Configure Nginx to serve static assets, proxy API routes, handle 50MB PDF statement uploads, and enable Let's Encrypt TLS 1.3:

```
sudo tee /etc/nginx/sites-available/ca-agent > /dev/null <<'EOF'
server {
    listen 80;
    server_name ca.yourdomain.com;
    # Allow large bank statement PDF/Image uploads
    client_max_body_size 50M;
    # High-performance Gzip compression
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml application/xml;
    # Frontend Single Page Application
    location / {
        root /var/www/ca-agent/dist;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }
    # API Proxy pass to FastAPI Backend
    location /api/ {
        proxy_pass http://127.0.0.1:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 180s;
        proxy_connect_timeout 60s;
    }
    # Proxy root level endpoints (/upload, /ask, /itr, /calendar, /docs)
    location ~ ^/(upload|ask|itr|calendar|docs|openapi.json) {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 180s;
        proxy_connect_timeout 60s;
    }
}
EOF
# Enable site, test configuration and reload Nginx
sudo ln -sf /etc/nginx/sites-available/ca-agent /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
# Issue free automatic SSL certificate via Let's Encrypt Certbot
sudo certbot --nginx -d ca.yourdomain.com --non-interactive --agree-tos -m admin@yourdomain.com
```

Step 3.7: Firewall Hardening (UFW)

```
# Allow SSH, HTTP, and HTTPS traffic only
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw --force enable
sudo ufw status
```

4. Method 2: Containerized Deployment (Docker & Docker Compose)

For portable cloud deployments (AWS ECS, Google Cloud Run, Azure Container Apps, or Docker Swarm), use this production multi-container topology comprising an **Ollama embedding service**, a **Python 3.11-slim FastAPI backend**, and an **Nginx Alpine frontend**.

4.1 Backend Dockerfile (Dockerfile.backend)

```
# Stage 1: Build wheel cache
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt
# Stage 2: Minimal runtime image
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1
# Copy source code and precomputed model matrices
COPY . .
RUN mkdir -p /app/temp && chmod 777 /app/temp
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=10s --start-period=45s --retries=3 \
    CMD curl -f http://localhost:8000/docs || exit 1
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

4.2 Frontend Dockerfile (Dockerfile.frontend)

```
# Stage 1: Build React static assets
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ENV VITE_API_URL=/api
RUN npm run build
# Stage 2: Serve via Nginx Alpine
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.docker.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4.3 Docker Compose Orchestrator (docker-compose.yml)

```
version: '3.8'
services:
  ollama:
    image: ollama/ollama:latest
    container_name: ca_ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama_storage:/root/.ollama
    entrypoint: >
      sh -c "ollama serve & sleep 5 && ollama pull bge-m3 && wait"
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    container_name: ca_backend
    restart: unless-stopped
    depends_on:
      - ollama
    ports:
      - "8000:8000"
    environment:
      - GEMINI_API_KEY=${GEMINI_API_KEY}
      - DEV_GEMINI_API_KEY=${DEV_GEMINI_API_KEY}
      - OLLAMA_URL=http://ollama:11434
      - ENABLE_HYDE=false
      - ENABLE_METRICS=false
    volumes:
      - temp_data:/app/temp
  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    container_name: ca_frontend
    restart: unless-stopped
    depends_on:
      - backend
    ports:
      - "80:80"
volumes:
  ollama_storage:
  temp_data:
```

4.4 Container Launch and Operations

```
# 1. Create .env with Gemini API keys
cat <<EOF > .env
GEMINI_API_KEY=your_actual_gemini_key
DEV_GEMINI_API_KEY=your_actual_dev_gemini_key
EOF
# 2. Build and launch all 3 microservices
docker compose up --build -d
# 3. Verify container health status
docker compose ps
docker compose logs -f backend
```

5. Method 3: Hybrid Cloud PaaS (Vercel + Render / Railway)

For teams wanting managed infrastructure without server maintenance, deploy the React frontend on **Vercel** and the FastAPI backend on **Render** or **Railway**.

5.1 Frontend Deployment on Vercel

1. Push your repository to GitHub.
2. In Vercel, click **Add New > Project** and import the repository.
3. Configure Build Settings: Framework Preset Vite, Build Command npm run build, Output Directory dist.
4. Set Environment Variable: VITE_API_URL=https://ca-backend.onrender.com.
5. Include a vercel.json file in the repository root to handle SPA client-side route rewrites:

```
{
  "rewrites": [
    { "source": "/api/(.*)", "destination": "https://ca-backend.onrender.com/api/$1" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

5.2 Backend Deployment on Render / Railway

1. In Render, create a new **Web Service** pointing to the repo.
2. Runtime: **Docker** (uses Dockerfile.backend) or **Python 3.11**.
3. Plan: Select an instance with at least **4 GB RAM** to accommodate the ICAI model embeddings.
4. Set Environment Variables: GEMINI_API_KEY, DEV_GEMINI_API_KEY.
5. **Ollama in Cloud PaaS**: Either connect to a persistent external Ollama instance by setting OLLAMA_URL or run Ollama on a lightweight cloud compute instance (AWS t3.medium / Hetzner CPX21).

6. Critical Production Hardening & Code Adjustments

During a comprehensive code audit of CA Agent 2.0, two critical deployment nuances were identified that must be addressed for cloud production:

Code Audit Finding: Hardcoded Endpoints in ITR Pages

While ChatContext.tsx properly honors import.meta.env.VITE_API_URL, two ITR pages contain hardcoded local URLs:
• src/pages/ITRFilingPage.tsx (lines 35, 48): uses http://localhost:8000/upload and /api/itr/analyze
• src/pages/ITRChatPage.tsx (lines 166, 189): uses http://localhost:8000/upload and /api/itr/chat
Remedy: Update these fetch calls to prefix with import.meta.env.VITE_API_URL || '' so they work in cloud deployments.

6.1 Production CORS Lockdown (api.py)

Replace the permissive wildcard origin in api.py with verified production domains:

```
# In api.py, replace lines 36-42 with:
ALLOWED_ORIGINS = [
    "https://ca.yourdomain.com",
    "https://ca-agent-frontend.vercel.app",
    "http://localhost:8080",  # Staging
]
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["*"],
)
```

6.2 Automated Storage Cleanup (temp/ Directory)

Add a daily cron job to prevent disk exhaustion from uploaded bank statements in temp/:

```
# Run crontab -e and add:
0 2 * * * find /var/www/ca-agent/temp -type f -mtime +1 -delete
```

7. Smoke Testing & Verification Suite

Execute these five automated tests to verify that RAG retrieval, Gemini Vision, ML Anomaly Detection, and ITR state machine are operating properly:

Test 1: Hybrid RAG Statutory Q&A (/ask)

```
curl -s -X POST https://ca.yourdomain.com/ask \
  -H "Content-Type: application/json" \
  -d '{
    "question": "What is the tax rate under Section 115BAC for income up to 7 lakhs?",
    "history": [],
    "doc_data": []
  }' | jq .answer
```

Test 2: Document Vision Extraction & ML Anomaly Flagging (/upload)

```
curl -s -X POST https://ca.yourdomain.com/upload \
  -F "file=@Statement.pdf" \
  -F "context=FY 2024-25" | jq '{person: .doc_item.person_entity, anomalies: .doc_item.flagged_transactions}'
```

Test 3: Guided ITR Conversational Agent (/api/itr/chat)

```
curl -s -X POST https://ca.yourdomain.com/api/itr/chat \
  -H "Content-Type: application/json" \
  -d '{
    "message": "I am a salaried individual with capital gains from equity shares. Which ITR should I file?",
    "history": []
  }' | jq .reply
```

Test 4: Tax Compliance Calendar Generator (/calendar)

```
curl -s -X POST https://ca.yourdomain.com/calendar \
  -H "Content-Type: application/json" \
  -d '{"doc_data": {}}' | jq .deadlines
```

8. Operational Troubleshooting Matrix

Quick reference guide for resolving common production issues:

Symptom / Error	Root Cause	Corrective Action
Embedding service (Ollama) is not running	Ollama process stopped or port 11434 unreachable.	Run sudo systemctl restart ollama and confirm ollama list shows bge-m3.
FileNotFoundError: embeddings_*.joblib	Git LFS pointers checked out instead of actual binaries.	Execute git lfs pull in the root directory to download all 3 corpora files (~270 MB total).
HTTP 413 Request Entity Too Large on PDF upload	Nginx default client body limit (1MB) exceeded.	Add client_max_body_size 50M; inside the Nginx server {} block and run nginx -s reload.
google.api_core.exceptions.ResourceExhausted: 429	Google Gemini API quota or rate limit exceeded.	Ensure paid tier is active on Google AI Studio or configure exponential backoff in ca_agent.py.
404 Not Found when refreshing page on route /itr-hub	Nginx missing SPA client route fallback.	Ensure Nginx has try_files $uri $uri/ /index.html; inside the location / block.
Ollama embedding timed out (30s timeout)	Model cold start on constrained CPU.	Increase timeout=60 in search.py:122 and ensure server has at least 4 vCPUs.

Deployment Complete & Operational!

Your CA Agent 2.0 platform is now fully deployed, hardened with TLS, and operational. For additional roadmap updates (Step 5 fine-tuned embeddings, Step 9 compliance calendar alerts, Step 10 voice interface), refer to PROJECT_CONTEXT.md.